TASK 4: CUSTOMER ORDER HISTORY AND ORDER MANAGEMENT SYSTEM

PROJECT DETAILS

- **Name:** Kaviya S
- **Roll No:** E25AI118
- **Department:** B.Sc Computer Science With AI

SCENARIO

The e-commerce bookstore (**PALETTE: Pixel & Pages**) wants to maintain information about customer transactions and order fulfillment. The system manages overall order records (such as order ID, customer ID, order date, total amount, and order status) and itemized line details (such as order detail ID, order ID, product ID, quantity, and price) to track purchasing history, monitor sales performance, and generate comprehensive sales analytics.

1. DESIGN ORDERS AND ORDER_DETAILS TABLES

i. Orders Table:

```
SQL> CREATE TABLE Orders (
  2      Order_ID VARCHAR2(10),
  3      Customer_ID VARCHAR2(20),
  4      Order_Date DATE,
  5      Total_Amount NUMBER(10,2),
  6      Order_Status VARCHAR2(20)
  7  );

Table created.
```

ii. Order_Details Table:

```
SQL> CREATE TABLE Order_Details (
  2      Order_Detail_ID VARCHAR2(10),
  3      Order_ID VARCHAR2(10),
  4      Product_ID VARCHAR2(7),
  5      Quantity NUMBER(5),
  6      Price NUMBER(9,2)
  7  );

Table created.
```

The Orders and Order_Details tables have been created successfully. Now, we will define the relationships between these tables and apply necessary constraints to enforce data integrity.

2. DEFINE RELATIONSHIPS AND CONSTRAINTS

i. Relationships:

- One Customer can place many Orders (1 → *).
- Customer_ID is the Primary Key in the customer table and acts as a Foreign Key in the Orders table to link the two entities.
- One Order can contain many Order_Details items (1 → *). Order_ID is the Primary Key in the Orders table and acts as a Foreign Key in the Order_Details table.
- Product_ID acts as a Foreign Key referencing the parent Book table.

ii. Constraint Explanations:

- **PRIMARY KEY:** Uniquely identifies each row in a table (Order_ID and Order_Detail_ID).
- **FOREIGN KEY:** Links the Orders table back to the parent customer table (Customer_ID), and Order_Details to Orders (Order_ID) and Book (Product_ID).
- **NOT NULL:** Guarantees that essential fields like Order_Date, Customer_ID, Quantity, and Price cannot be left empty.
- **CHECK:** Validates that Quantity and Price are strictly positive (>0) and restricts Order_Status to valid workflow states ('Pending', 'Shipped', 'Delivered', 'Cancelled').

iii. Defining Relationship and Creating Constraints for Orders and Order_Details:

a. Primary Key Constraints

```
SQL> ALTER TABLE Orders ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);

Table altered.

SQL> ALTER TABLE Order_Details ADD CONSTRAINT PK_Order_Details PRIMARY KEY (Order_Detail_ID);

Table altered.
```

b. Foreign Key Constraints

```
SQL> ALTER TABLE Orders ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES customer (customer_id);

Table altered.

SQL> ALTER TABLE Order_Details ADD CONSTRAINT FK_OD_Orders FOREIGN KEY (Order_ID) REFERENCES Orders (Order_ID);

Table altered.

SQL> ALTER TABLE Order_Details ADD CONSTRAINT FK_OD_Book FOREIGN KEY (Product_ID) REFERENCES Book (Book_ID);

Table altered.
```

iv. Adding Constraints:

a. NOT NULL Constraints

```
SQL> ALTER TABLE Orders MODIFY Order_Date NOT NULL;

Table altered.

SQL> ALTER TABLE Orders MODIFY Customer_ID NOT NULL;

Table altered.

SQL> ALTER TABLE Order_Details MODIFY Quantity NOT NULL;

Table altered.

SQL> ALTER TABLE Order_Details MODIFY Price NOT NULL;

Table altered.
```

b. CHECK Constraints

```
SQL> ALTER TABLE Orders ADD CONSTRAINT CHK_Order_Status CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered', 'Cancelled'));

Table altered.

SQL> ALTER TABLE Order_Details ADD CONSTRAINT CHK_Quantity CHECK (Quantity > 0);

Table altered.
```

```
SQL> ALTER TABLE Order_Details ADD CONSTRAINT CHK_OD_Price CHECK (Price > 0);

Table altered.
```

3. INSERT VALUES

i. Orders Table Interactive Insert:

```
SQL> INSERT INTO Orders VALUES ('&Order_ID', '&Customer_ID', TO_DATE('&Order_Date', 'YYYY-MM-DD'), &Total_Amount, '&Order_Status');
Enter value for order_id: ORD-501
Enter value for customer_id: CUST-101
Enter value for order_date: 2026-03-01
Enter value for total_amount: 1098.00
Enter value for order_status: Delivered
old   1: INSERT INTO Orders VALUES ('&Order_ID', '&Customer_ID', TO_DATE('&Order_Date', 'YYYY-MM-DD'), &Total_Amount, '&Order_Status'
)
new   1: INSERT INTO Orders VALUES ('ORD-501', 'CUST-101', TO_DATE('2026-03-01', 'YYYY-MM-DD'), 1098.00, 'Delivered')

1 row created.

SQL> /
Enter value for order_id: ORD-502
Enter value for customer_id: CUST-101
Enter value for order_date: 2026-03-05
Enter value for total_amount: 1250.00
Enter value for order_status: Delivered
old   1: INSERT INTO Orders VALUES ('&Order_ID', '&Customer_ID', TO_DATE('&Order_Date', 'YYYY-MM-DD'), &Total_Amount, '&Order_Status'
)
new   1: INSERT INTO Orders VALUES ('ORD-502', 'CUST-101', TO_DATE('2026-03-05', 'YYYY-MM-DD'), 1250.00, 'Delivered')

1 row created.

SQL> /
Enter value for order_id: ORD-503
Enter value for customer_id: CUST-102
Enter value for order_date: 2026-03-10
Enter value for total_amount: 599.00
Enter value for order_status: Pending
old   1: INSERT INTO Orders VALUES ('&Order_ID', '&Customer_ID', TO_DATE('&Order_Date', 'YYYY-MM-DD'), &Total_Amount, '&Order_Status'
)
new   1: INSERT INTO Orders VALUES ('ORD-503', 'CUST-102', TO_DATE('2026-03-10', 'YYYY-MM-DD'), 599.00, 'Pending')

1 row created.
```

```
SQL> /
Enter value for order_id: ORD-504
Enter value for customer_id: CUST-103
Enter value for order_date: 2026-03-12
Enter value for total_amount: 1149.00
Enter value for order_status: Shipped
old   1: INSERT INTO Orders VALUES ('&Order_ID', '&Customer_ID', TO_DATE('&Order_Date', 'YYYY-MM-DD'), &Total_Amount, '&Order_Status'
)
new   1: INSERT INTO Orders VALUES ('ORD-504', 'CUST-103', TO_DATE('2026-03-12', 'YYYY-MM-DD'), 1149.00, 'Shipped')

1 row created.
```

ii. Order_Details Table Interactive Insert:

```
SQL> INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price);
Enter value for order_detail_id: OD-01
Enter value for order_id: ORD-501
Enter value for product_id: B1001
Enter value for quantity: 1
Enter value for price: 499.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-01', 'ORD-501', 'B1001', 1, 499.00)

1 row created.

SQL> /
Enter value for order_detail_id: OD-02
Enter value for order_id: ORD-501
Enter value for product_id: B1002
Enter value for quantity: 1
Enter value for price: 599.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-02', 'ORD-501', 'B1002', 1, 599.00)

1 row created.

SQL> /
Enter value for order_detail_id: OD-03
Enter value for order_id: ORD-502
Enter value for product_id: B1004
Enter value for quantity: 1
Enter value for price: 1250.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-03', 'ORD-502', 'B1004', 1, 1250.00)

1 row created.
```

```
SQL> /
Enter value for order_detail_id: OD-04
Enter value for order_id: ORD-503
Enter value for product_id: B1002
Enter value for quantity: 1
Enter value for price: 599.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-04', 'ORD-503', 'B1002', 1, 599.00)

1 row created.

SQL> /
Enter value for order_detail_id: OD-05
Enter value for order_id: ORD-504
Enter value for product_id: B1001
Enter value for quantity: 1
Enter value for price: 499.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-05', 'ORD-504', 'B1001', 1, 499.00)

1 row created.

SQL> /
Enter value for order_detail_id: OD-06
Enter value for order_id: ORD-504
Enter value for product_id: B1003
Enter value for quantity: 1
Enter value for price: 650.00
old   1: INSERT INTO Order_Details VALUES ('&Order_Detail_ID', '&Order_ID', '&Product_ID', &Quantity, &Price)
new   1: INSERT INTO Order_Details VALUES ('OD-06', 'ORD-504', 'B1003', 1, 650.00)

1 row created.
```

iii. Display Tables:

a. Orders Table

```
SQL> SELECT * FROM Orders;

ORDER_ID   CUSTOMER_ID          ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- -------------------- --------- ------------ --------------------
ORD-501    CUST-101             01-MAR-26         1098 Delivered
ORD-502    CUST-101             05-MAR-26         1250 Delivered
ORD-503    CUST-102             10-MAR-26          599 Pending
ORD-504    CUST-103             12-MAR-26         1149 Shipped
```

b. Order_Details Table

```
SQL> SELECT * FROM Order_Details;

ORDER_DETA ORDER_ID   PRODUCT   QUANTITY      PRICE
---------- ---------- ------- ---------- ----------
OD-01      ORD-501    B1001            1        499
OD-02      ORD-501    B1002            1        599
OD-03      ORD-502    B1004            1       1250
OD-04      ORD-503    B1002            1        599
OD-05      ORD-504    B1001            1        499
OD-06      ORD-504    B1003            1        650

6 rows selected.
```

4. GENERATE CUSTOMER ORDER HISTORY REPORTS

i. Report 1: Customer Order History

- **Display:**
  - Customer name
  - Order ID
  - Order date
  - Total amount
  - Status

```
SQL> SELECT
  2      c.first_name || ' ' || c.last_name AS Customer_Name,
  3      o.Order_ID,
  4      o.Order_Date,
  5      o.Total_Amount,
  6      o.Order_Status AS Status
  7  FROM customer c
  8  JOIN Orders o ON c.customer_id = o.Customer_ID;

CUSTOMER_NAME                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
-------------------------------------- ---------- --------- ------------
STATUS
--------------------
Ananya Roy                             ORD-501    01-MAR-26         1098
Delivered

Ananya Roy                             ORD-502    05-MAR-26         1250
Delivered

Rohan Mehta                            ORD-503    10-MAR-26          599
Pending


CUSTOMER_NAME                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
-------------------------------------- ---------- --------- ------------
STATUS
--------------------
Priya Nair                             ORD-504    12-MAR-26         1149
Shipped
```

ii. Report 2: Product-wise Order Report

- **Display:**
  - Product name
  - Number of times each product was ordered
  - Total quantity sold

```
SQL> SELECT
  2      b.Title AS Product_Name,
  3      COUNT(od.Order_ID) AS Times_Ordered,
  4      SUM(od.Quantity) AS Total_Quantity_Sold
  5  FROM Book b
  6  JOIN Order_Details od ON b.Book_ID = od.Product_ID
  7  GROUP BY b.Title;

PRODUCT_NAME
--------------------------------------------------------------------------------
TIMES_ORDERED TOTAL_QUANTITY_SOLD
------------- -------------------
Dune
            2                   2

Foundation
            1                   1

Designing Data-Intensive Applications
            1                   1


PRODUCT_NAME
--------------------------------------------------------------------------------
TIMES_ORDERED TOTAL_QUANTITY_SOLD
------------- -------------------
The Great Gatsby
            2                   2
```

iii. Report 3: Customer Purchase Analysis

- **Find:**
  - **Customers with maximum orders**

```
SQL> SELECT
  2      c.first_name || ' ' || c.last_name AS Customer_Name,
  3      COUNT(o.Order_ID) AS Total_Orders
  4  FROM customer c
  5  JOIN Orders o ON c.customer_id = o.Customer_ID
  6  GROUP BY c.customer_id, c.first_name, c.last_name
  7  HAVING COUNT(o.Order_ID) = (
  8      SELECT MAX(COUNT(Order_ID))
  9      FROM Orders
 10      GROUP BY Customer_ID
 11  );

CUSTOMER_NAME                          TOTAL_ORDERS
-------------------------------------- ------------
Ananya Roy                                        2
```

- **Total spending by each customer**

```
SQL> SELECT
  2      c.first_name || ' ' || c.last_name AS Customer_Name,
  3      SUM(o.Total_Amount) AS Total_Spending
  4  FROM customer c
  5  JOIN Orders o ON c.customer_id = o.Customer_ID
  6  GROUP BY c.customer_id, c.first_name, c.last_name;

CUSTOMER_NAME                          TOTAL_SPENDING
-------------------------------------- --------------
Rohan Mehta                                       599
Ananya Roy                                       2348
Priya Nair                                       1149
```

- **Average order value**

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Value
  2  FROM Orders;

AVERAGE_ORDER_VALUE
-------------------
               1024
```